



UNIVERSIDAD
TECNOLÓGICA
METROPOLITANA
del Estado de Chile

INFORME TRABAJO ANÁLISIS

COMPUTACIÓN PARALELA Y DISTRIBUIDA

Nombres:

Matias Ziade

Julián Vidal Jorquera

Víctor Morales Cavieres

Fecha:

10 de julio de 2026

Sección:

411

Profesor:

Sebastián Salazar Molina

Índice

Resumen ejecutivo	2
1. Descripción de la solución implementada	2
1.1 Arquitectura general	2
1.2 Pipeline de ejecución	3
1.3 Reproducibilidad	4
1.4 Estructura del repositorio	4
2. Justificación de las librerías estadísticas utilizadas	4
3. Metodología y resultados	6
3.1 Preprocesamiento y limpieza de datos	6
3.1.1 Carga paralela y particionado	6
3.1.2 Benchmark de paralelismo: cuándo SÍ y cuándo NO conviene paralelizar	6
3.1.3 Reglas de limpieza aplicadas	8
3.1.4 Detección de outliers (IQR y Z-score)	8
3.1.5 Variables derivadas	10
3.1.6 Hallazgo: UNIDADES es constante	10
3.1.7 Arquitectura híbrida Dask + Pandas	10
3.2 Análisis exploratorio estadístico	11
3.2.1 Estadística descriptiva	11
3.2.2 Tests de normalidad	12
3.2.3 Correlación y asociación	17
3.2.4 Chi-cuadrado: CANAL vs LOCAL	18
3.2.5 Kruskal-Wallis (equivalente no paramétrico de ANOVA)	19
3.2.6 Patrones temporales	19
3.2.7 Comparación de fechas especiales, controlada por día de la semana	22
3.3 Inferencia estadística y modelado	23
3.3.1 Hipótesis del enunciado, Ejemplo 1: ticket promedio APP vs WEB	23
3.3.2 Hipótesis del enunciado, Ejemplo 2: descuento vs. cantidad comprada	24
3.3.3 Tres hipótesis propias	25
3.3.4 Modelo de regresión	26
Diagnósticos del modelo	28
4. Dificultades encontradas y cómo se resolvieron	29
5. Limitaciones y extrapolabilidad	32
Trabajo futuro	32
6. Conclusiones	33
7. Referencias teóricas	34

Resumen ejecutivo

El presente informe técnico detalla los resultados de un análisis estadístico avanzado y de modelado predictivo aplicado al histórico transaccional de la farmacia Cruz Morada. El estudio exigió el procesamiento de un volumen masivo de datos, compuesto por un dataset de 634 MB con más de 3.24 millones de registros. Para superar los cuellos de botella computacionales y optimizar el rendimiento, se implementó una arquitectura de procesamiento paralelo híbrida: se utilizó Dask para la ingesta y particionado en bloques, Pandas para las operaciones complejas en memoria, y DuckDB para ejecutar consultas SQL analíticas de alta velocidad sobre un Data Lake almacenado en formato columnar Parquet. Se garantizó la reproducibilidad estricta del entorno fijando parámetros deterministas.

Tras las fases de limpieza, imputación y validación de supuestos estadísticos, la aplicación de pruebas no paramétricas y regresión lineal multivariada (OLS) arrojó los siguientes hallazgos críticos para la toma de decisiones:

- **Equidad en canales digitales:** No existe una diferencia estadísticamente significativa ni de impacto práctico en el ticket promedio de compra entre las plataformas APP y WEB.
- **Determinismo de infraestructura:** El canal de venta se encuentra casi estrictamente determinado por la sucursal física, evidenciándose que la práctica totalidad de los locales operan bajo un único entorno comercial.
- **Asimetría de descuentos:** Los canales remotos presentan incentivos más agresivos, otorgando en promedio un 11% más de descuento respecto a las compras en punto de venta (POS).
- **Límites de extrapolación:** El modelo predictivo construido logra explicar el 25% de la varianza en el monto de ventas ($R^2 = 0,25$), confirmando que el valor base de los artículos, variable omitida dada su altísima cardinalidad, es el factor predominante

1. Descripción de la solución implementada

1.1 Arquitectura general

La solución se estructura en tres capas: (1) una capa de paralelismo para la carga y el almacenamiento de datos, basada en Dask; (2) una capa de almacenamiento analítico en formato Parquet, organizada como un *data lake* particionado por la columna LOCAL; y (3) una capa de consulta y análisis, que combina DuckDB (consultas SQL embebidas sobre Parquet, sin servidor) con Pandas, SciPy y statsmodels para el cómputo estadístico.

El dataset (634 MB, 3.242.878 filas) cabe en memoria con los 16 GB de RAM disponibles, por lo que Dask se utiliza específicamente donde aporta valor demostrable, lectura paralela por bloques y escritura particionada, mientras que las transformaciones que requieren *shuffle* entre particiones (agrupaciones por cliente o por boleta, que no coinciden con el particionado por LOCAL) se resuelven materializando a Pandas. Esta decisión de arquitectura híbrida está documentada y cuantificada en la sección 3.1.

1.2 Pipeline de ejecución

El pipeline se compone de 11 scripts numerados, ejecutados en orden por un orquestador (main.py) que además soporta ejecución parcial (--desde N, --solo N) para facilitar la depuración incremental.

#	Script	Función
01	01_load_and_partition.py	Carga paralela del CSV con Dask y partición a Parquet por LOCAL
02	02_cleaning_and_features.py	Limpieza de datos y creación de variables derivadas
03	03_descriptive_stats.py	Estadística descriptiva vía consultas SQL en DuckDB
04	04_histograms_normality.py	Histogramas, boxplots y tests de normalidad (Shapiro-Wilk, Lilliefors)
05	05_association_analysis.py	Correlación de Spearman, chi-cuadrado, Kruskal-Wallis
06	06_temporal_patterns.py	Series de tiempo: descomposición STL, ACF/PACF, patrón semanal
07	07_special_dates_local_baseline.py	Comparación de fechas especiales controlada por día de semana
08	08_hypothesis_testing.py	Pruebas de hipótesis del enunciado (Ejemplo 1 y Ejemplo 2)
09	09_own_hypotheses.py	Tres hipótesis propias
10	10_regression_model.py	Modelo de regresión OLS con diagnósticos completos

11	11_outlier_detection.py	Detección de outliers (IQR y Z-score)
----	-------------------------	---------------------------------------

Tabla: Pipeline de 11 etapas, ejecutado secuencialmente por main.py.

1.3 Reproducibilidad

Toda la aleatoriedad del proyecto (muestreos para tests de normalidad, partición train/test, semillas de numpy) se controla mediante una única semilla fija, leída desde la variable de entorno CPYD_SEED (valor por defecto: 42), definida en un archivo .env y cargada con python-dotenv. Cada script acepta además un argumento --seed para sobreescribirla puntualmente por línea de comandos.

1.4 Estructura del repositorio

```
cruzmorada/
├── .env                # CPYD_SEED=42
├── .gitignore
├── README.md
├── requirements.txt
├── main.py            # Orquestador del pipeline
├── data/
│   ├── ventas_completas.csv (no versionado)
│   ├── lake/          # Parquet crudo, particionado por LOCAL
│   └── lake_clean/     # Parquet limpio + variables derivadas
├── src/
│   ├── 01_load_and_partition.py
│   ├── ... (02 a 11)
│   └── exploración/    # Scripts de diagnóstico (no forman parte del pipeline)
└── reports/figures/   # Gráficos PNG y tablas CSV de resultados
```

2. Justificación de las librerías estadísticas utilizadas

Librería	Rol en el proyecto	Por qué esta y no otra
Dask	Lectura por bloques, particionado y escritura paralela del CSV/Parquet	Recomendada por el enunciado para paralelismo; API casi idéntica a Pandas, minimiza riesgo de reescritura de lógica
DuckDB	Consultas SQL analíticas embebidas directamente sobre Parquet	Sin servidor, sin carga completa a memoria; funciones de agregación estadística nativas

		(SKEWNESS, KURTOSIS, QUANTILE_CONT) muy eficientes sobre millones de filas
scipy.stats	Tests: Shapiro-Wilk, Mann-Whitney U, Kruskal-Wallis, Spearman, chi-cuadrado	Implementación estándar, ampliamente validada, de los tests clásicos de inferencia
statsmodels	OLS, VIF, Breusch-Pagan, Lilliefors, descomposición STL, ACF/PACF	A diferencia de scikit-learn, entrega errores estándar, p-valores e intervalos de confianza para los coeficientes, imprescindible para la interpretación inferencial que pide el enunciado
pingouin	Correlaciones por pares con p-value e IC 95% en un solo llamado (pairwise_corr)	Evita calcular manualmente cada par de correlaciones y su significancia
scikit-learn	train_test_split, métricas RMSE/MAE	Utilidades estándar de partición y evaluación, sin necesidad de implementarlas a mano
matplotlib / seaborn	Visualización: histogramas con KDE, heatmaps, boxplots	Seaborn simplifica la sintaxis estadística (KDE, boxplots agrupados) sobre la base de matplotlib
pandas	Manipulación de DataFrames tras materializar desde Dask	Estándar de facto; necesario para las operaciones de groupby/join no paralelizables eficientemente
pyarrow	Motor de serialización Parquet	Motor recomendado y más maduro para Parquet en el ecosistema Python/Dask

3. Metodología y resultados

3.1 Preprocesamiento y limpieza de datos

3.1.1 Carga paralela y particionado

El archivo se carga con `dask.dataframe.read_csv` en bloques de 64 MB, forzando el tipado explícito de cada columna para evitar que Dask infiera los tipos leyendo el archivo completo dos veces. Un hallazgo temprano fue que el separador real del CSV es punto y coma (;), con todos los campos entre comillas dobles, no coma, como se asumió inicialmente (ver sección 4, dificultad #1). Tras la carga, el dataset se escribe como Parquet particionado por LOCAL (792 valores únicos).

```
ddf = dd.read_csv(
    str(input_path), dtype=COLUMN_DTYPES,
    blocksize=args.blocksize, # 64MB por defecto
    encoding='utf-8', sep=args.sep, # ';', confirmado con debug_lines.py
    quotechar="\"",
)
ddf = ddf.rename(columns=RENAME_MAP)
...
ddf.to_parquet(
    str(output_dir), partition_on=['local'],
    engine='pyarrow', write_index=False,
)
```

Validación de corrección: se contrastó el conteo de filas leído directamente del CSV crudo (vía DuckDB) contra el conteo de filas en el Parquet particionado resultante, obteniendo exactamente el mismo número en ambos casos (3.242.878 filas), confirmando que la lectura y el particionado paralelos no alteran ni pierden datos respecto de una lectura secuencial completa.

3.1.2 Benchmark de paralelismo: cuándo SÍ y cuándo NO conviene paralelizar

Metodología de medición: los tiempos se midieron con `time.time()` (*wall-clock*) de la biblioteca estándar de Python, sobre una muestra fija de 300.000 filas (parámetro `--benchmark-rows`), comparando una lectura serial con Pandas (1 core) contra una lectura paralela con Dask, ambas leyendo el mismo archivo bajo el mismo entorno virtual (venv aislando dependencias) y las mismas condiciones de sistema.

IMPORTANTE: [COMPLETAR: modelo de CPU, número de núcleos físicos/lógicos, sistema operativo y versión exacta de Python usados para esta medición].

Escenario	Herramienta	Tiempo medido	Observación
Muestra de 300.000 filas	Pandas (serial, 1 core)	0,82 s	—
Muestra de 300.000 filas	Dask (paralelo, 41 particiones)	8,34 s	10 veces más lento que Pandas
Dataset completo (3,24M filas)	Dask, lectura + partición Parquet	31,19 s	Incluye escritura a disco

Interpretación no técnica: en la muestra chica, Dask resultó diez veces más lento que Pandas. Esto no es un error, es el costo de coordinación del paralelismo: construir el grafo de tareas y repartir el trabajo entre particiones tiene un costo fijo que, para 300.000 filas, supera ampliamente el tiempo que se ahorra paralelizando. El paralelismo sólo compensa cuando el volumen de trabajo por partición es suficientemente grande.

Este mismo principio se confirmó de forma mucho más dramática al construir las variables derivadas (sección 3.1.4): intentar resolver agrupaciones por cliente y por boleta usando Dask puro, con datos particionados por LOCAL (una clave no relacionada), no terminó tras más de 20 minutos (47% de avance). El mismo cálculo, materializando a Pandas antes de agrupar, tomó 86,9 segundos, una mejora de más de 14× que ilustra el mismo principio: el paralelismo distribuido tiene overhead de coordinación que solo se amortiza cuando la clave de partición coincide con la clave de la operación.

3.1.3 Reglas de limpieza aplicadas

#	Regla	Detalle	Acción	Filas afectadas
1	PCT_DESCUENTO fuera de [0,1]	Un descuento >100% o negativo es imposible	Eliminación de la fila (error de registro)	1
2	EDAD implícita fuera de [0,100]	(fecha – fecha_nacimiento) / 365,25	NaN → imputación con mediana (48 años)	3.056
3	Año de nacimiento corrupto	Años fuera de [1900,2024], p.ej. "9771", "7968", "0960"	Pre-filtrado como string antes de parsear a fecha	149
4	Duplicados boleto+sku+cliente+fecha	401.122 filas comparten esta clave, pero difieren en monto/descuento	Se conservan, no son duplicados reales (ver 4.4)	0 eliminadas

En total, 3.205 filas (3.056 + 149) recibieron una edad imputada con la mediana del resto del dataset. Se utilizó la mediana en lugar de la media porque es más robusta ante distribuciones con outliers o colas asimétricas, que en este dataset es la norma, no la excepción.

Justificación del mecanismo de falta de datos (MCAR): el enunciado sugiere, como ejemplo, justificar el tratamiento de datos faltantes con una prueba como el test de MCAR. En este caso se optó por una justificación narrativa en lugar de un test formal: los años de nacimiento corruptos (p.ej. "9771-11-08", "0960-09-27") tienen la apariencia de errores de tipeo o corrupción aislada durante la generación del dataset, no se observó ningún patrón de asociación aparente entre estos registros y otras variables como CANAL o LOCAL, lo que es consistente con un mecanismo de pérdida completamente al azar (MCAR) y por lo tanto respalda la imputación simple por mediana como estrategia adecuada.

3.1.4 Detección de outliers (IQR y Z-score)

Además de las reglas de limpieza basadas en dominio (tabla anterior), se aplicó una detección de outliers con dos métodos robustos estándar sobre las variables numéricas continuas, sin modificar los datos, sólo como diagnóstico para determinar si los valores extremos son errores de registro o casos de negocio reales.

Variable	Límites IQR	Outliers IQR	% IQR	Outliers Z-score ($ Z >3$)	% Z-score
monto_aplicado	[-4.775,5 ; 20.612,5]	212.494	6,55%	39.973	1,23%
edad	[-5,5 ; 102,5]	0	0,00%	128	0,004%
pct_descuento	[0,125 ; 0,725]	13.668	0,42%	13.503	0,42%
frecuencia_compra_cliente	[-4 ; 12]	112.094	3,46%	52.481	1,62%

Interpretación: para monto_aplicado, el IQR marca un 6,55% de outliers mientras que el Z-score solo un 1,23%, la brecha entre ambos métodos es esperable dada la asimetría extrema de la variable (skewness=9,06): el IQR se basa en cuartiles (poco sensibles a la cola), mientras que la desviación estándar usada por el Z-score ya está inflada por esa misma cola pesada, haciendo al método más permisivo. Determinación de negocio: estos outliers de monto muy probablemente NO son errores. El modelo de regresión (sección 3.3.4) muestra que el monto está fuertemente asociado al nivel de descuento aplicado, que a su vez actúa como proxy del precio/categoría del producto, así, montos muy altos son consistentes con productos de alto costo real, no con datos corruptos. edad no presenta outliers relevantes (ya se trató en la limpieza de dominio); pct_descuento y frecuencia_compra_cliente presentan outliers minoritarios, coherentes con clientes muy frecuentes o niveles de descuento poco comunes, también casos de negocio reales.

3.1.5 Variables derivadas

Variable	Definición	Nota
edad	$(\text{fecha_transacción} - \text{fecha_nacimiento}) / 365,25$, en años	Se usa 365,25 (no 365) para no sesgar por años bisiestos
monto_por_unidad	monto_aplicado / unidades	Idéntico a monto_aplicado, ya que UNIDADES es siempre 1 (ver 3.1.6)
frecuencia_compra_cliente	nunique(boleta) agrupado por codigo_cliente	Corregido durante la revisión de calidad del código: no count(boleta), ver sección 4, dificultad #8
unidades_producto_boleta	count(*) agrupado por (boleta, sku)	Reconstruye la cantidad real comprada por producto en cada transacción
monto_aplicado_z, edad_z	Estandarización Z-score con media/std globales	Solo para EDA, el modelo de regresión no las usa (evita fuga de datos)

3.1.6 Hallazgo: UNIDADES es constante

Se determinó que la columna UNIDADES tiene valor 1 en el 100% de las 3.242.877 filas del dataset limpio. Cada fila representa la venta de una unidad física individual, no una línea de venta con cantidad variable. Esto se confirmó junto con otro hallazgo relacionado: los 401.122 casos de filas que comparten boleta+sku+cliente+fecha no son duplicados, son unidades individuales del mismo producto vendidas en la misma boleta, cada una con un descuento distinto (posiblemente un esquema de descuento escalonado por unidad). Se verificó que 0 filas son duplicados exactos (las 15 columnas originales idénticas). Para recuperar la noción de "cantidad comprada", se construyó unidades_producto_boleta agrupando por (boleta, sku), variable que sí presenta variabilidad real (rango 1 a 4).

3.1.7 Arquitectura híbrida Dask + Pandas

Las variables derivadas que requieren groupby + join (frecuencia_compra_cliente, unidades_producto_boleta) se calculan materializando el DataFrame a Pandas, no en Dask puro. La razón es doble: (1) el dataset cabe cómodamente en los 16 GB de RAM disponibles, y (2) las claves de agrupación (codigo_cliente, boleta+sku) no coinciden con el particionado del data lake por LOCAL, lo que fuerza a Dask a hacer un shuffle costoso entre las 792 particiones, operación que, medida empíricamente, no terminó en más de 20 minutos. El mismo cálculo con el enfoque híbrido (lectura y escritura en Dask, transformación en Pandas) tomó 86,9 segundos en total. Esta es una decisión de ingeniería explícita y medida, no una renuncia al paralelismo: Dask se sigue usando donde

efectivamente aporta (E/S paralela), y se evita donde el overhead de coordinación supera el beneficio.

3.2 Análisis exploratorio estadístico

3.2.1 Estadística descriptiva

Calculada íntegramente con consultas SQL nativas de DuckDB directamente sobre los archivos Parquet, sin cargar el dataset completo a un DataFrame en memoria:

```
SELECT
  COUNT(monto_aplicado)          AS n,
  ROUND(AVG(monto_aplicado), 4)   AS media,
  ROUND(MEDIAN(monto_aplicado), 4) AS mediana,
  ROUND(STDDEV(monto_aplicado), 4) AS desv_std,
  ROUND(QUANTILE_CONT(monto_aplicado,0.25),4) AS q1,
  ROUND(QUANTILE_CONT(monto_aplicado,0.75),4) AS q3,
  ROUND(SKEWNESS(monto_aplicado), 4)   AS asimetria,
  ROUND(KURTOSIS(monto_aplicado), 4)   AS curtosis
FROM 'data/lake_clean/**/*'.parquet'
```

Variable	n	Media	Mediana	Desv. Std	Mín	Máx	Asimetría	Curtosis
monto_aplicado	3.242.877	10.179,98	7.662,0	14.453,24	15	226.476	9,06	108,23
edad	3.242.877	49,06	48,0	16,72	0	100	0,24	-0,79
pct_descuento	3.242.877	0,392	0,4	0,108	0	1,0	0,30	2,85
frecuencia_compra_cliente	3.242.877	4,50	4,0	3,76	1	97	2,79	24,86
unidades_producto_boleta	3.242.877	1,13	1,0	0,37	1	4	2,74	7,30

Canal	n	Media monto	Mediana monto	Desv. std monto
POS	3.119.427	10.146,77	7.595,0	14.488,27
WEB	79.986	11.167,75	8.379,0	13.910,72
APP	43.238	10.750,44	8.379,0	12.751,11
CCT	226	9.765,00	7.745,0	10.077,31

Interpretación no técnica: monto_aplicado y frecuencia_compra_cliente están muy sesgados a la derecha (asimetría 9,06 y 2,79 respectivamente), hay muchas ventas de monto moderado y pocas ventas muy grandes, que estiran el promedio hacia arriba (por eso la media de monto, \$10.180, es mayor que la mediana, \$7.662). edad, en cambio, tiene una forma bastante simétrica y cercana a lo normal (asimetría 0,24). WEB tiene el ticket promedio más alto entre los canales, seguido de APP, POS y CCT.

3.2.2 Tests de normalidad

Decisión de muestreo: los tests se corrieron sobre una muestra aleatoria fija de $n=5.000$ (semilla CPYD_SEED), no sobre las 3,24 millones de filas completas. Con un N tan grande, cualquier desviación mínima de la normalidad produce p-valores prácticamente cero, el test se vuelve hipersensible y deja de ser informativo (diferencia entre significancia estadística y relevancia práctica).

Kolmogorov-Smirnov vs. Lilliefors: se utilizó el test de Lilliefors en lugar del test de Kolmogorov-Smirnov (KS) clásico. Cuando los parámetros de la distribución de referencia (media, desviación estándar) se estiman a partir de la misma muestra que se está testeando como es el caso aquí, las tablas de valores críticos del KS estándar dejan de ser válidas y producen p-valores sistemáticamente demasiado conservadores. Lilliefors es la corrección estándar de este problema, con tablas críticas específicas para ese escenario, y es la forma correcta de aplicar, en la práctica, el test que el enunciado nombra genéricamente como "Kolmogorov-Smirnov".

Variable	Shapiro-Wilk (stat)	Shapiro-Wilk (p)	Lilliefors (stat)	Lilliefors (p)	¿Normal? ($\alpha=0,05$)
monto_aplicado	0,360	$1,4 \times 10^{-85}$	0,268	<0,001	No
edad	0,979	$1,9 \times 10^{-26}$	0,056	<0,001	No
pct_descuento	0,847	$1,2 \times 10^{-56}$	0,188	<0,001	No
frecuencia_compra_cliente	0,791	$2,0 \times 10^{-62}$	0,177	<0,001	No
unidades_producto_boleta	0,394	$2,3 \times 10^{-84}$	0,518	<0,001	No

Interpretación no técnica: ninguna de las cinco variables numéricas sigue una distribución normal ($p < 0,05$ en ambos tests para todas). edad es la más cercana a la normalidad (estadístico de Shapiro-Wilk=0,979, el más próximo a 1), aunque técnicamente también se rechaza. Esta conclusión determina el resto de las decisiones metodológicas del análisis: se usan tests no paramétricos (Spearman, Mann-Whitney, Kruskal-Wallis) en lugar de sus equivalentes paramétricos (Pearson, t-test, ANOVA) en todo el resto del informe.

Figura 1. Distribución de edad.

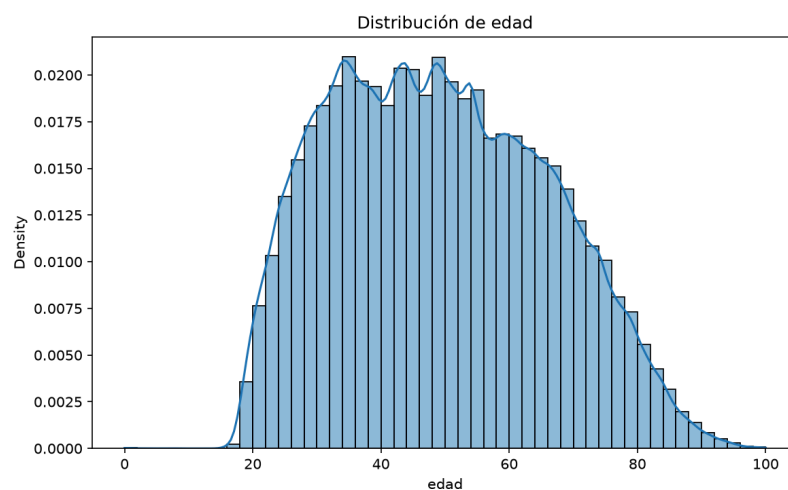


Figura 2. Distribución de monto_aplicado (escala original).

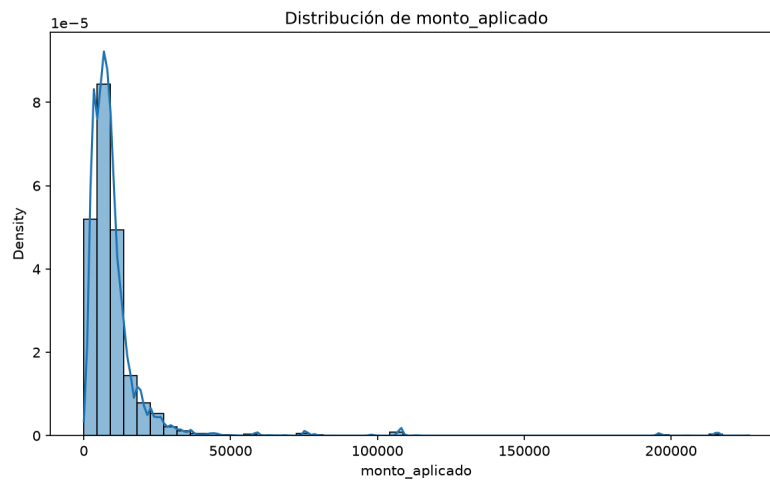
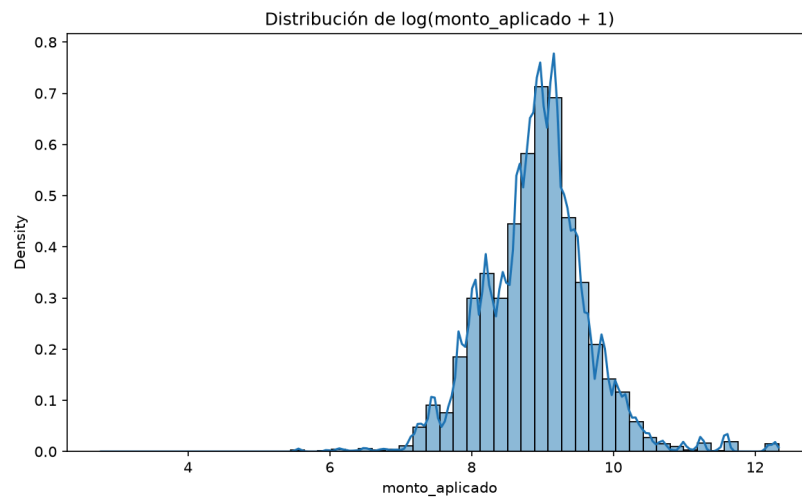


Figura 3. Distribución de $\log(\text{monto_aplicado} + 1)$.



La comparación entre las Figuras 2 y 3 justifica la transformación logarítmica usada más adelante en el modelo de regresión (sección 3.3.4): en escala original la distribución es una cola larga casi irreconocible; en escala logarítmica se aproxima mucho más a una campana.

Figura 4. Distribución de pct_descuento (multimodal: niveles de descuento comerciales discretos, no continuos).

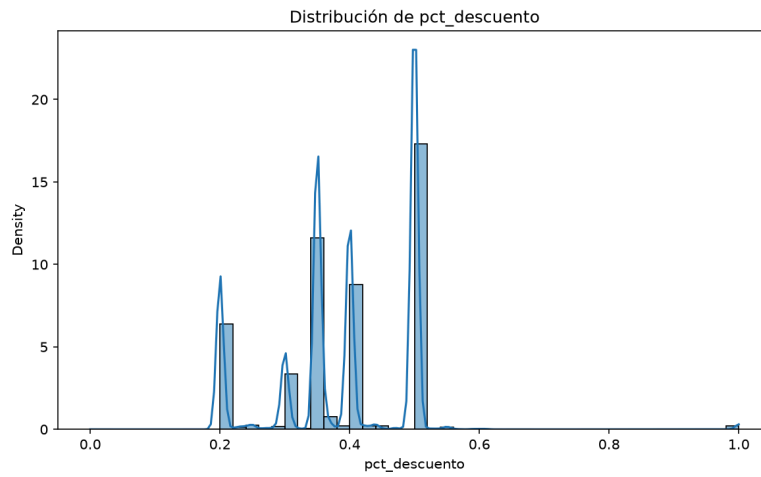


Figura 5. Distribución de unidades_producto_boleta (esencialmente discreta: 1 a 4).

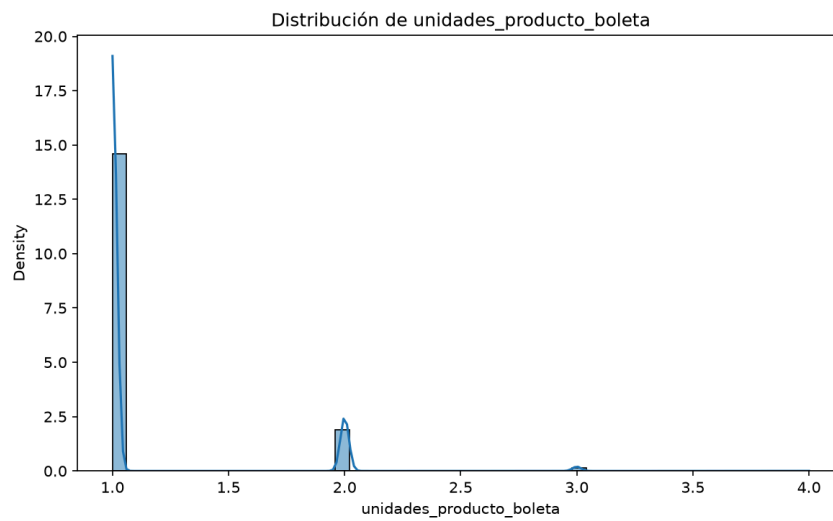


Figura 6. monto_aplicado por canal (outliers extremos ocultos para legibilidad).

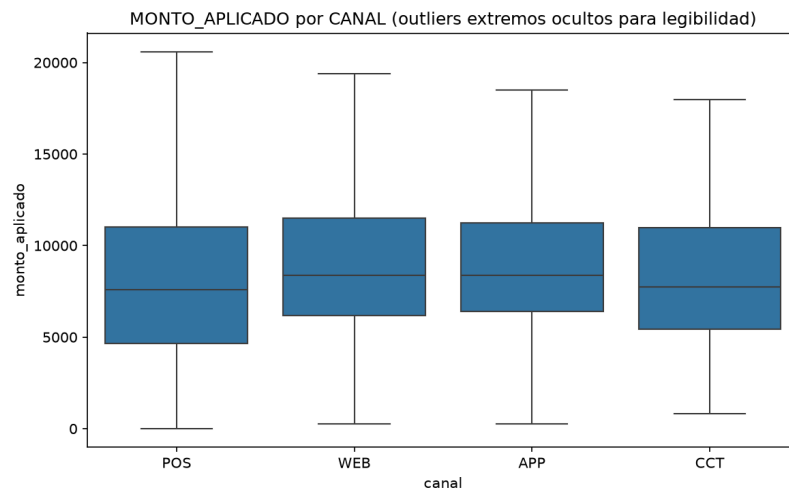
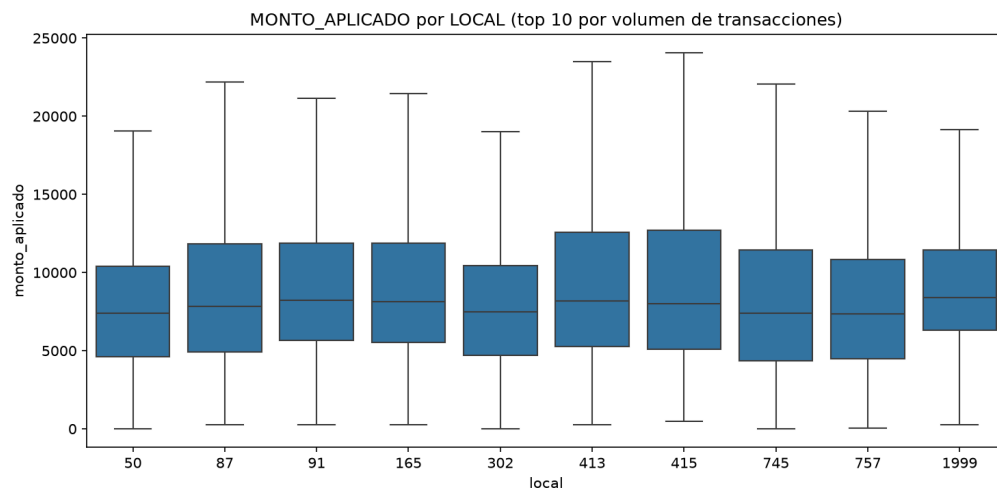


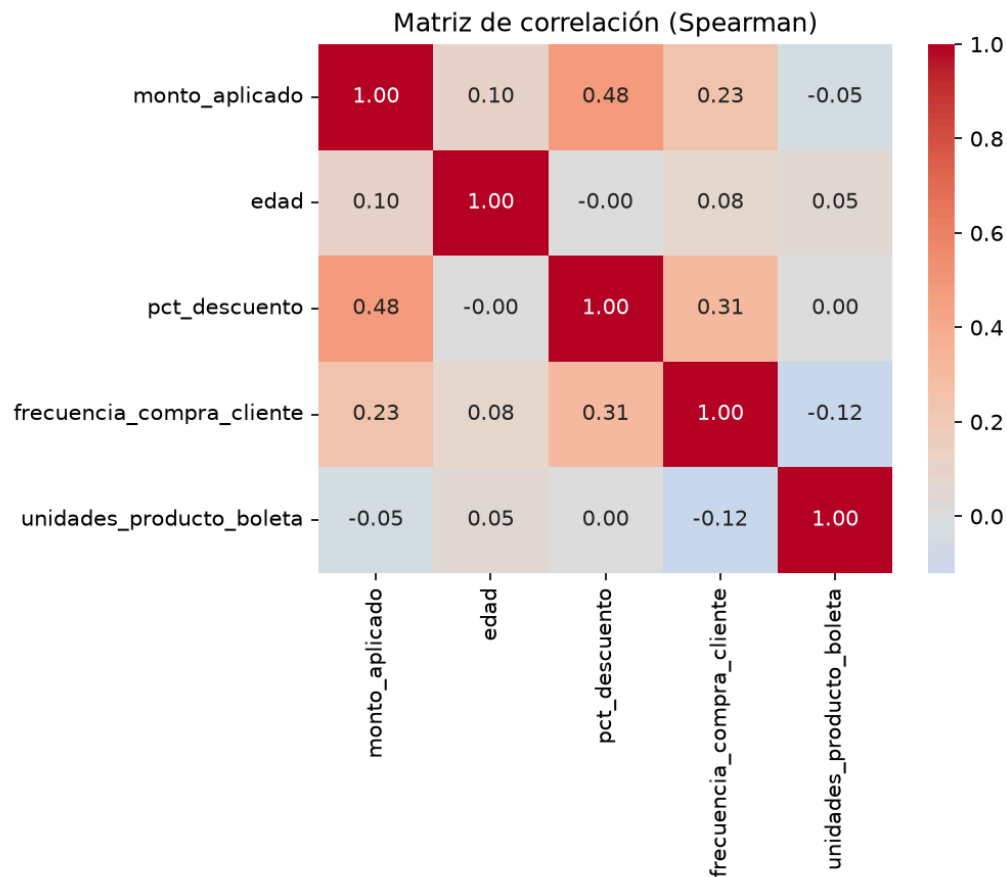
Figura 7. monto_aplicado por local (top 10 por volumen de transacciones).



3.2.3 Correlación y asociación

Pearson vs. Spearman: se utilizó exclusivamente la correlación de Spearman (basada en rangos), no la de Pearson. Esta decisión es consecuencia directa del punto 3.2.2: ninguna variable pasó los tests de normalidad, condición necesaria para que Pearson sea la elección apropiada. Spearman es robusto ante distribuciones no normales y captura relaciones monótonas no necesariamente lineales.

Figura 8. Matriz de correlación de Spearman.



Par de variables	ρ (rho)	p-valor	Comentario
monto_aplicado, pct_descuento	0,483	≈ 0	La correlación más fuerte del análisis
monto_aplicado / frecuencia_compra_cliente	0,230	≈ 0	Débil

monto_aplicado / edad	0,099	≈ 0	Muy débil
monto_aplicado / unidades_producto_boleta	-0,049	≈ 0	Prácticamente nula
pct_descuento / frecuencia_compra_cliente	0,305	≈ 0	Débil-moderada
edad / pct_descuento	-0,004	$2,0 \times 10^{-14}$	Nula en la práctica pese al p-valor
pct_descuento / unidades_producto_boleta	0,0002	0,741	Único par NO significativo

Nota metodológica clave: con $N=3.242.877$, casi cualquier correlación distinta de cero resulta "estadísticamente significativa", incluso cuando el coeficiente ρ es tan pequeño que carece de relevancia práctica (p.ej. edad, pct_descuento: $\rho=-0,004$ con $p=2 \times 10^{-14}$). Por eso, en todo este informe se reporta y discute el tamaño de efecto (ρ , r , R^2 , Cramér's V, ϵ^2) junto al p-valor, nunca el p-valor de forma aislada.

3.2.4 Chi-cuadrado: CANAL vs LOCAL

Versión	χ^2	gl	p	% celdas con esperado < 5	Cramér's V
Completa (792 locales)	3.242.877,00	2.373	0	25,7%	-
Agregada (top 10 + OTROS)	3.242.877,00	30	0	20,5%	0,577

Hallazgo principal del análisis exploratorio: el estadístico χ^2 es idéntico (e igual al total de filas, N) tanto en la versión completa como en la agregada, un patrón matemáticamente inusual que motivó una verificación directa: se calculó, para cada uno de los 792 locales, qué porcentaje de sus transacciones caen en su canal más frecuente. El resultado: 791 de 792 locales (99,9%) operan bajo un solo canal, prácticamente todos corresponden a POS. El único local mixto (identificador 1999) concentra el 100% de las transacciones de los canales no presenciales: APP (43.238), CCT (226) y WEB (79.986). En otras palabras, LOCAL determina CANAL de forma casi perfecta. Esta es la razón por la que LOCAL se

excluye como predictor del modelo de regresión (sección 3.3.4): incluir ambas variables generaría multicolinealidad severa sin aportar información adicional.

3.2.5 Kruskal-Wallis (equivalente no paramétrico de ANOVA)

Se usó Kruskal-Wallis en lugar de ANOVA de una vía por el mismo motivo que Spearman reemplaza a Pearson: ninguna variable es normal.

Comparación	H	p
monto_aplicado por CANAL	5.950,65	0
monto_aplicado por LOCAL (top 10)	2.598,82	0

3.2.6 Patrones temporales

Hallazgo crítico: la serie diaria completa tiene 0 transacciones entre el 10 de noviembre de 2023 y el 6 de abril de 2024 (149 días consecutivos, con solo 8 transacciones el primer día del dataset). Esto no representa un cierre real de la farmacia, sino un artefacto de cómo se generó el dataset sintético, a partir del 7 de abril de 2024 la actividad crece de forma muy pronunciada (de ~10 transacciones/día a más de 30.000/día hacia fines de 2024), un ritmo de crecimiento poco realista para un negocio real. En consecuencia, todo el análisis de series de tiempo se restringe al período con actividad consistente (7 de abril a 2 de diciembre de 2024, 240 días), de lo contrario, el bloque inicial de ceros distorsiona por completo la descomposición STL.

Figura 9. Monto total de ventas por día, período confiable (desde 2024-04-07).

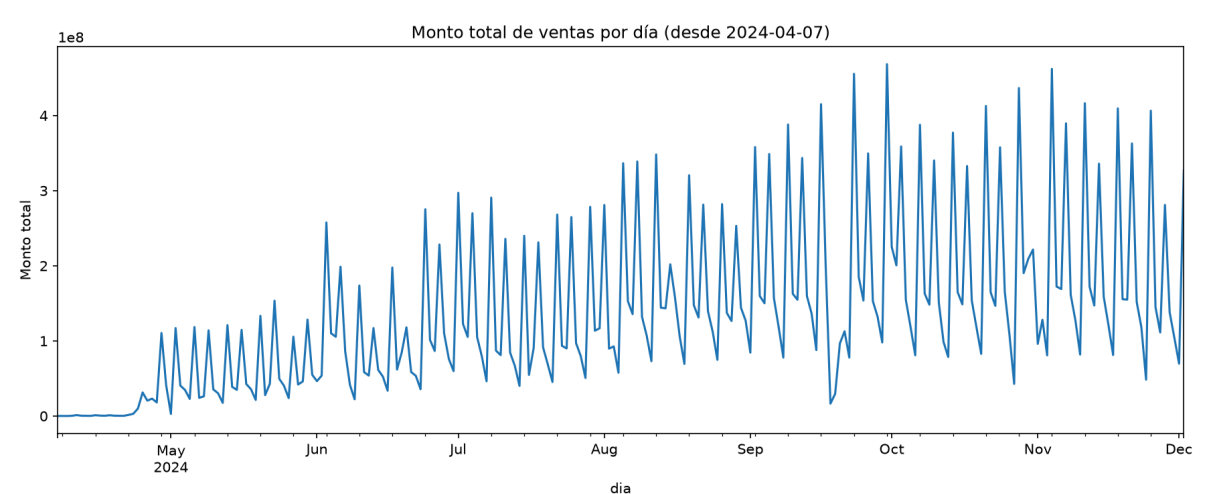


Figura 10. Descomposición STL (tendencia, estacionalidad, residuo), period=7, modo robusto.

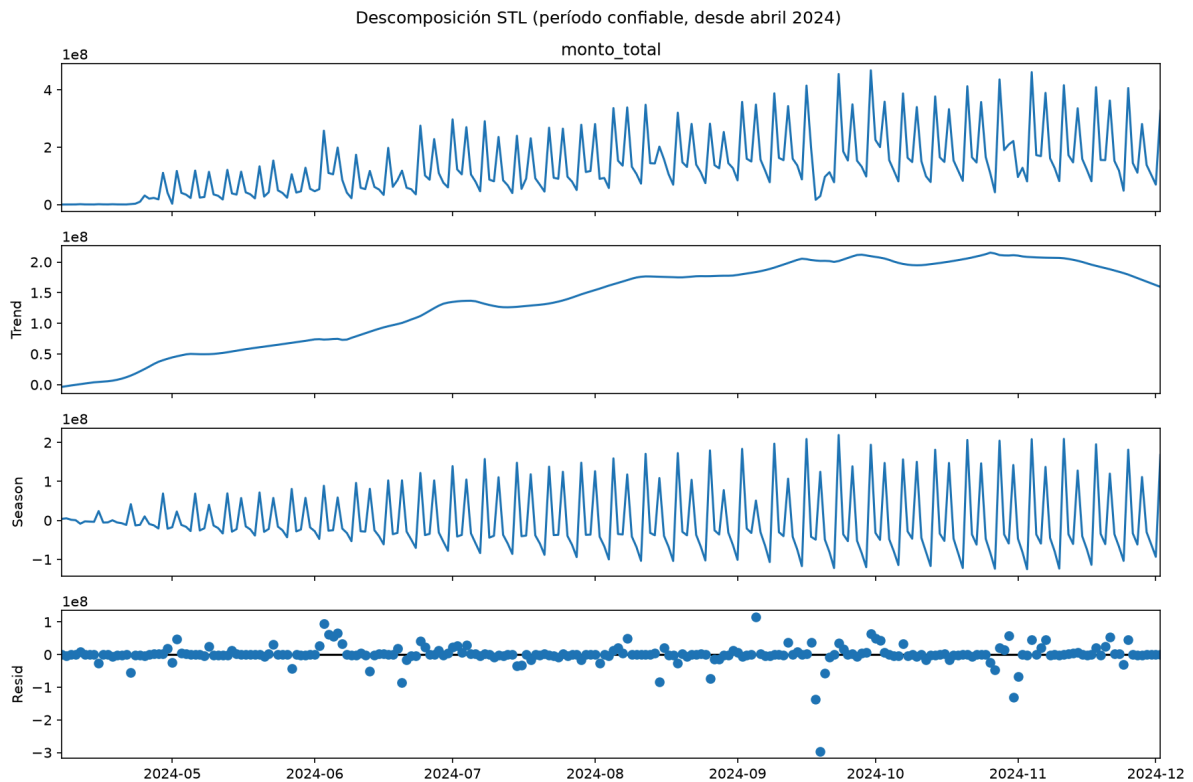
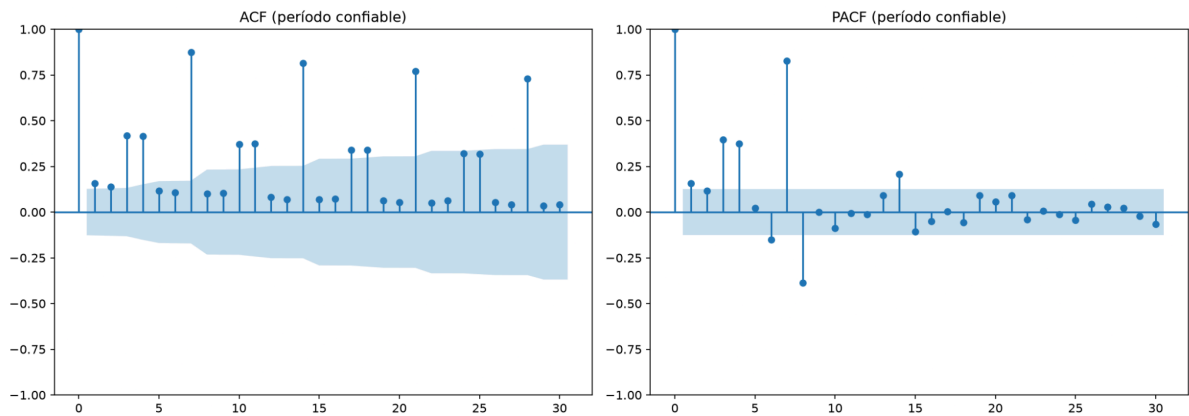
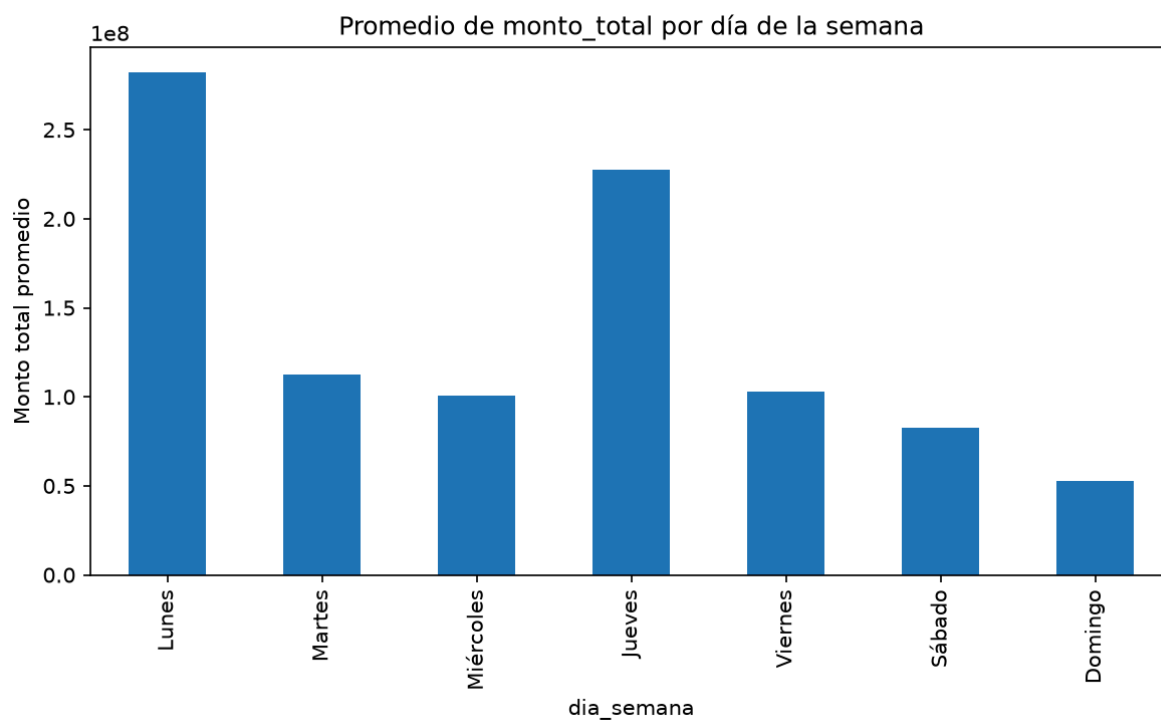


Figura 11. ACF y PACF (30 lags), período confiable.



Tanto el ACF como el PACF muestran picos claros y decrecientes en los lags 7, 14, 21 y 28, evidencia directa de una estacionalidad semanal marcada. El patrón de promedio de ventas por día de la semana confirma y explica esta estructura:

Figura 12. Promedio de monto_total por día de la semana.



Día	Monto total promedio
Lunes	\$282.230.100
Jueves	\$227.459.900
Martes	\$112.372.500
Viernes	\$102.915.100
Miércoles	\$100.747.500
Sábado	\$82.850.000
Domingo	\$52.529.650

Interpretación: lunes y jueves son los días pico (explican también los picos secundarios en lag 3-4 del ACF: lunes→jueves son 3 días, jueves→lunes siguiente son 4). La magnitud de la diferencia entre el día pico y el mínimo (más de 5×) es inusualmente grande para un patrón de retail típico (normalmente 20-50%), lo que sugiere que el patrón semanal real de negocio probablemente se mezcla con algún artefacto adicional de la generación sintética del dataset, se reporta tal cual, con esta salvedad.

3.2.7 Comparación de fechas especiales, controlada por día de la semana

Comparar una fecha especial contra el promedio de todo el período confunde dos efectos simultáneos: el día de la semana en que cae esa fecha (p.ej. el Día de la Madre 2024 cayó sábado-domingo, ya de por sí los días de menor venta) y la fuerte tendencia de crecimiento de la serie. Para aislar el efecto real de la fecha, cada una se comparó contra el promedio del mismo día de la semana en una ventana de ± 3 semanas (excluyendo la fecha en cuestión).

Fecha	Día	Valor real	Baseline (mismo día, ± 3 sem.)	Variación	Comparación ingenua (todo el período)
Día de la Madre (11 may 2024)	Sábado	\$30.418.336	\$30.121.172	+1,0%	-82,6%
Día de la Madre (12 may 2024)	Domingo	\$17.553.475	\$23.360.582	-24,9%	-82,6%
Fiestas Patrias (18 sep 2024)	Miércoles	\$16.543.200	\$155.651.946	-89,4%	-83,2%
Fiestas Patrias (19 sep 2024)	Jueves	\$29.559.902	\$332.349.155	-91,1%	-83,2%

Interpretación no técnica: el aparente "efecto" del Día de la Madre (-82,6% frente al promedio general) desaparece casi por completo al comparar correctamente (sábado real +1,0%, domingo real -24,9%), no hay evidencia sólida de un repunte o caída real de ventas de farmacia por esta fecha. Fiestas Patrias, en cambio, muestra una caída real y muy fuerte (-89% a -91%), incluso mayor que la comparación ingenua, porque el jueves 18 de septiembre 2024 es normalmente uno de los días de mayor venta (jueves pico, ver 3.2.6) y cayó a casi nada, consistente con cierre o actividad muy reducida por feriado nacional.

Observación: No fue posible incluir una comparación de Navidad: el 24-25 de diciembre de 2023 cae dentro del período sin datos (sección 3.2.6), y el dataset termina el 2 de diciembre de 2024, antes de Navidad de ese año. Se documenta como limitación en la sección 5.

3.3 Inferencia estadística y modelado

3.3.1 Hipótesis del enunciado, Ejemplo 1: ticket promedio APP vs WEB

H_0 : no hay diferencia en el ticket promedio entre APP y WEB. H_1 : sí hay diferencia. Test principal: Mann-Whitney U (no paramétrico, dado que monto_aplicado no es normal). Se reporta también un t-test de Welch como comparación.

Grupo	n	Media	Mediana
APP	43.238	10.750,44	8.379,00
WEB	79.986	11.167,75	8.379,00

Test	Estadístico	p-valor	Decisión ($\alpha=0,05$)
Mann-Whitney U (principal)	1.731.339.763,00	0,7217	No se rechaza H_0
t-test de Welch (comparación)	$t=-5,3085$	$1,11 \times 10^{-7}$	Se rechaza H_0
Tamaño de efecto (r biserial de rango)	-0,0012	—	Efecto prácticamente nulo

Interpretación no técnica: el p-valor de Mann-Whitney (0,72) no rechaza H_0 : los canales APP y WEB tienen ventas centralmente similares (de hecho, la mediana es idéntica en ambos: \$8.379). El t-test, en cambio, sí resulta significativo ($p<0,001$), pero es un resultado engañoso: monto_aplicado tiene asimetría extrema (9,06), y el t-test compara promedios, que son sensibles a la cola pesada de la distribución (unas pocas transacciones muy grandes en WEB, que tiene casi el doble de transacciones que APP, alcanzan para desplazar el promedio sin mover la tendencia central real). El tamaño de efecto de Mann-Whitney ($r=-0,0012$) confirma que, aunque existiera una diferencia, sería del todo irrelevante en la práctica. Conclusión: no hay evidencia de que el ticket promedio realmente difiera entre APP y WEB.

3.3.2 Hipótesis del enunciado, Ejemplo 2: descuento vs. cantidad comprada

H_0 : la pendiente entre el descuento y la cantidad comprada es igual a cero. H_1 : el descuento afecta significativamente la cantidad. Se evaluó a dos niveles de agregación distintos, como ejercicio de robustez metodológica.

Nivel de agregación	n	Spearman ρ (p)	Coef. OLS (pendiente)	p (pendiente)	R ²
Línea de producto/unidad (unidades_producto_boleta)	3.242.877	0,0002 (p=0,741)	-0,0417	<0,001	0,00015
Boleta agregada (suma de unidades por boleta)	2.769.573 boletas	0,0262 (p≈0)	+0,0476	<0,001	0,00012

Interpretación no técnica: a nivel de línea de producto, la correlación de Spearman no es significativa (p=0,74): no hay relación aparente. A nivel de boleta agregada, la correlación sí es significativa y con signo positivo, coincidiendo con el resultado obtenido de forma independiente por otro grupo del curso sobre el mismo tipo de datos, usando el mismo nivel de agregación (verificación cruzada). El cambio de signo entre ambos niveles se explica por pseudo-replicación: a nivel de línea de producto, una boleta con varias unidades del mismo producto aporta varias observaciones con el mismo total de unidades pero descuentos individuales distintos por unidad, diluyendo o distorsionando la relación real. En ambos niveles, sin embargo, el R² es prácticamente cero (0,00012 a 0,00015): el descuento explica, como máximo, el 0,015% de la variación en la cantidad comprada. Conclusión de fondo: aunque técnicamente "significativo" en algunas especificaciones (producto de un N de millones de observaciones), el descuento no tiene un efecto de relevancia práctica sobre la cantidad comprada.

3.3.3 Tres hipótesis propias

H1 : ¿El monto promedio difiere según el género del cliente?

Grupo	n	Media	Mediana
Género 1	1.055.885	10.698,75	7.876,00
Género 2	2.186.992	9.929,51	7.495,00

Mann-Whitney $U=1.209.371.782.034,00$, $p \approx 0$ (se rechaza H_0). Tamaño de efecto $r=-0,047$ (pequeño, pero el más grande observado en todo el análisis inferencial). El género 1 presenta un ticket ~7% más alto que el género 2, una diferencia real, aunque modesta.

H2 : ¿Hay asociación entre la edad del cliente y su frecuencia de compra?

Nota metodológica: esta correlación se calculó a nivel de CLIENTE (una fila por codigo_cliente, $n=1.183.242$), no a nivel de transacción. Calcularla sobre las 3,24 millones de filas de transacciones habría pesado a un cliente con 97 compras 97 veces más que a uno con una sola compra, sesgando el resultado hacia los clientes más frecuentes (pseudo-replicación).

Spearman $\rho=0,0192$, $p=1,33 \times 10^{-96}$. Estadísticamente significativo (con más de un millón de clientes, incluso un p casi nulo se vuelve "significativo"), pero sin ninguna relevancia práctica: la edad prácticamente no se asocia con la frecuencia de compra.

H3 : ¿El descuento promedio varía según el canal de venta?

Canal	Media	Mediana	Desv. std
APP	0,4335	0,5	0,0825
CCT	0,4256	0,4	0,0774
POS	0,3906	0,4	0,1085
WEB	0,4221	0,4	0,0914

Kruskal-Wallis $H=16.533,90$, $p \approx 0$ (se rechaza H_0). Este es el hallazgo más relevante en términos de negocio de las tres hipótesis propias: POS presenta un descuento promedio notablemente más bajo (0,391) que los tres canales no presenciales (0,42-0,43), una diferencia relativa de ~10-11%. Consistente con el hallazgo de la sección 3.2.4 (CANAL $\approx$ LOCAL): los canales remotos, que comparten el mismo local "virtual", parecen operar bajo una política de descuentos más generosa que las tiendas físicas.

3.3.4 Modelo de regresión

Especificación del modelo (Opción A del enunciado): $\log(\text{monto_aplicado}) \sim \text{canal} + \text{unidades_producto_boleta} + \text{pct_descuento}$

Decisión de diseño	Justificación
Transformación logarítmica de la variable dependiente	monto_aplicado tiene asimetría extrema (9,06). $\log(\text{monto_aplicado})$ se acerca mucho más a una normal (Figura 3) y, a diferencia de una winsorización (recorte de outliers), no descarta información real de transacciones de alto valor
LOCAL excluido como predictor	792 niveles, casi perfectamente confundido con CANAL (sección 3.2.4): incluir ambos generaría multicolinealidad severa sin aportar información nueva
Categoría de referencia: canal POS	Es el canal más frecuente (96,2% de las transacciones)
Split train/test 70/30 ANTES de ajustar el modelo	Previene fuga de datos
No se usan las columnas _z precalculadas	Se calculan con estadísticos globales (todo el dataset); el modelo construye sus features directamente de las columnas crudas

Split ANTES de ajustar cualquier estadístico, sin fuga de datos

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=args.seed,
)
```

```
X_train_const = sm.add_constant(X_train)
```

```
modelo = sm.OLS(y_train, X_train_const).fit()
```

```
# Cualquier estandarización adicional se ajusta sobre X_train
```

```
# y se aplica (nunca se re-ajusta) sobre X_test.
```

Entrenamiento: 2.270.013 filas. Prueba: 972.864 filas. $R^2=0,2524$, R^2 ajustado= $0,2524$, $F=1,532 \times 10^5$ ($p \approx 0$).

Variable	Coef. (log)	Error std.	t	p	Efecto % aprox.
const (intercepto)	7,7108	0,002	3742,0	<0,001	—
unidades_producto_boleta	-0,1106	0,001	-96,46	<0,001	-10,47%
pct_descuento	3,3744	0,004	865,20	<0,001	+2.820% ($\approx 29\times$)
canal_APP	0,0199	0,004	5,42	<0,001	+2,01%
canal_CCT	-0,0607	0,049	-1,24	0,216	No significativo
canal_WEB	0,0696	0,003	25,64	<0,001	+7,21%

Interpretación de coeficientes: unidades_producto_boleta (-10,5% por unidad adicional del mismo producto) es coherente con un esquema de descuento por volumen dentro de la misma boleta (sección 3.1.6). canal_WEB (+7,2%) y canal_APP (+2,0%) confirman, ya en un modelo multivariado, el orden ya visto en la descriptiva (3.2.1): WEB > APP > POS en ticket promedio. canal_CCT no es significativo ($p=0,216$), esperable con solo 226 observaciones. El coeficiente de pct_descuento (equivalente a un multiplicador de $\sim 29\times$ al pasar de 0% a 100% de descuento) es, con certeza, el hallazgo que exige más cuidado interpretativo de todo el informe: no significa que dar más descuento haga que el cliente pague más. La explicación más probable es una variable omitida, el precio original o la categoría del producto, que el dataset no incluye. Es plausible que los descuentos grandes (40-50%) se apliquen sistemáticamente a productos más caros (tratamientos especializados) y los descuentos chicos (20%) a productos baratos (genéricos), de forma que pct_descuento actúa como un proxy indirecto del precio del producto, no como una causa directa de un monto más alto. Esta observación es central para la discusión de extrapolabilidad (sección 5).

Diagnósticos del modelo

Variable	VIF
const	24,02 (no interpretable como colinealidad; artefacto normal del intercepto)
unidades_producto_boleta	1,002
pct_descuento	1,004
canal_APP	1,003
canal_CCT	1,000
canal_WEB	1,004

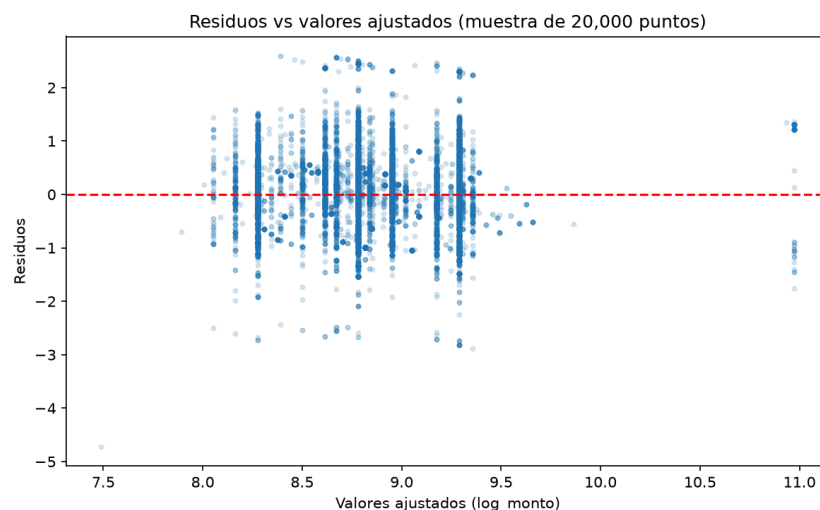
Todos los VIF de las variables explicativas están prácticamente en 1,0, no hay multicolinealidad relevante, lo cual confirma que excluir LOCAL fue la decisión correcta (con LOCAL incluido, el VIF de CANAL habría sido muy superior).

Diagnóstico	Resultado	Conclusión
Normalidad de residuos, Shapiro-Wilk (n=5.000)	W=0,9717, $p = 2,57 \times 10^{-30}$	Se rechaza; pero la asimetría de los residuos (0,224) mejoró radicalmente respecto de monto_aplicado crudo (9,06)
Normalidad de residuos, Lilliefors	stat=0,0582, $p=0,001$	Se rechaza
Homocedasticidad, Breusch-Pagan	LM=7710,42, $p \approx 0$	Se rechaza: hay heterocedasticidad

El gráfico de residuos vs. valores ajustados (generado por el script 10, reports/figures/residuos_vs_ajustados.png) complementa visualmente el test de Breusch-Pagan.

Evaluación en el conjunto de prueba: RMSE=0,6321 y MAE=0,4673 en escala logarítmica; RMSE≈\$12.579 y MAE≈\$4.843 en escala original (con una nota metodológica: al des-transformar $\exp(\log(\hat{y}))$ se introduce un sesgo leve y conocido por la desigualdad de Jensen, por lo que estas dos últimas cifras son aproximadas).

Figura 13. Residuos vs. Valores Ajustados (Modelo de regresión)



Como complemento visual a las pruebas formales, el gráfico de residuos frente a valores ajustados confirma lo detectado por el test de Breusch-Pagan. Se puede apreciar una clara dispersión no constante de los errores (heterocedasticidad) a medida que aumenta el valor predicho, lo que reafirma las limitaciones del modelo lineal básico y justifica la interpretación conservadora de sus coeficientes.

4. Dificultades encontradas y cómo se resolvieron

El desarrollo de este proyecto analítico con herramientas de procesamiento distribuido (Dask) y bases de datos analíticas (DuckDB) presentó una serie de desafíos técnicos y estadísticos. A través de un proceso iterativo de pruebas, validación cruzada de datos y una rigurosa revisión de código final, se lograron identificar y superar los siguientes obstáculos:

1. Separador de CSV mal identificado

La primera carga con `pandas.read_csv` (separador por defecto, coma) produjo un `ParserError` intermitente ("Expected 2 fields, saw 3"). Diagnóstico: se inspeccionaron líneas crudas del archivo (sin pasar por pandas) y se confirmó que el separador real es punto y coma (;), con todos los campos entre comillas dobles; las comas visibles pertenecían a nombres de producto con decimales europeos (p.ej. "12,5MG"), correctamente protegidas por las comillas. Solución: parámetro `--sep ';'` explícito en la lectura con Pandas y Dask.

2. Overhead de coordinación de Dask en muestras chicas

El benchmark serial vs. paralelo (sección 3.1.2) mostró a Dask 10× más lento que Pandas sobre una muestra de 300.000 filas. Diagnóstico: el costo fijo de construir el grafo de tareas y coordinar particiones supera el beneficio del paralelismo cuando el volumen de trabajo por partición es chico. Se documentó como hallazgo, no como error, es la razón detrás de la decisión de arquitectura híbrida.

3. Groupby/merge en Dask no terminaba (20+ minutos)

Al calcular las variables derivadas usando Dask puro, con datos particionados por LOCAL y agrupaciones por `codigo_cliente/boleta+sku` (claves no relacionadas con LOCAL), el proceso se estancó en 47% de avance tras más de 20 minutos. Diagnóstico: el shuffle entre 792 particiones no alineadas con las claves de agrupación es extremadamente costoso. Solución: materializar a Pandas para esas operaciones específicas (86,9 segundos), manteniendo Dask sólo para lectura/escritura.

4. Bug de `stats.kstest` con versión reciente de `scipy`

La llamada `stats.kstest(datos, "norm", args=(media, std))` producía `TypeError: "ndtr() takes from 1 to 2 positional arguments but 3 were given"`, incompatibilidad entre la API antigua de `kstest` y la versión de `scipy` instalada. Solución: usar una distribución congelada, `stats.kstest(datos, stats.norm(loc=media, scale=std).cdf)`, compatible con versiones recientes.

5. UNIDADES constante = 1 en el 100% de las filas

Al perfilar la columna UNIDADES se encontró `min=max=1` en todo el dataset, cada fila representa una unidad física vendida, no una línea de venta con cantidad variable. Esto invalidaba, tal como estaba planteada, la hipótesis del enunciado sobre el efecto del descuento en las unidades. Solución: reconstrucción de la cantidad real mediante `unidades_producto_boleta` (conteo agrupado por `boleta+sku`).

6. "401.122 duplicados" que resultaron ser unidades legítimas

Un chequeo inicial de duplicados por `boleta+sku+cliente+fecha` marcó 401.122 filas como posibles duplicados. Un chequeo más fino (comparando las 15 columnas completas) mostró 0 duplicados exactos: cada grupo tenía el mismo total de unidades pero montos y descuentos distintos por fila, son unidades individuales del mismo producto vendidas con descuentos diferentes dentro de la misma boleto, no duplicados. Se decidió conservarlas todas.

7. Overflow de datetime64 con años de nacimiento corruptos

Al convertir fecha_nacimiento a datetime con Dask, valores corruptos como "9771-11-08" o "7968-09-24" (fuera del rango representable por datetime64[ns], aprox. 1677-2262) producían OverflowError incluso con errors='coerce'. Solución: pre-validar el año como substring de texto (anio_str = fecha_nacimiento.str.slice(0,4)) y descartar como texto los años fuera de [1900,2024] antes de intentar la conversión a fecha, evitando construir el valor imposible.

8. Revisión exhaustiva y refactorización del código

Durante la etapa de validación del proyecto, realicé una revisión exhaustiva de la lógica de negocio y el código implementado, lo que me permitió detectar y corregir tres problemas metodológicos: (a) la métrica frecuencia_compra_cliente utilizaba inicialmente count(boleta) en vez de nunique(boleta), lo cual contabilizaba líneas de producto en lugar de visitas reales (una boletoa con 10 productos sumaba erróneamente 10 a la frecuencia); (b) el cálculo de edad utilizaba días // 365 en lugar de 365.25, lo que generaba un leve sesgo al ignorar los años bisiestos; (c) el test de Kolmogorov-Smirnov (KS) en los residuos del modelo presentaba el mismo problema detallado en la dificultad #4 (uso de parámetros estimados de la misma muestra), por lo que fue reemplazado por el test de Lilliefors. Los tres puntos se corrigieron y el data lake se regeneró desde cero. Tras esta corrección, se verificó que el cambio de signo en la correlación entre frecuencia_compra_cliente y unidades_producto_boleta (de +0,04 a -0,12) es ahora completamente coherente con la realidad del negocio: los clientes frecuentes compran menos unidades por visita.

9. Hueco de 149 días en la serie temporal

La serie diaria completa mostró 0 transacciones entre el 10 de noviembre de 2023 y el 6 de abril de 2024. Diagnóstico: artefacto de la generación sintética del dataset, no un cierre real. Solución: restringir todo el análisis de series de tiempo al período con actividad consistente (desde el 2024-04-07).

10. Comparación de fechas especiales confundida por el día de la semana

Una primera comparación de fechas especiales contra el promedio general del período mostró caídas similares (~-83%) tanto para el Día de la Madre como para Fiestas Patrias, lo cual resultó engañoso: el Día de la Madre 2024 cae sábado-domingo, días de por sí bajos. Solución: comparar cada fecha contra el promedio del mismo día de la semana en una ventana cercana (± 3 semanas), lo que reveló que el efecto real del Día de la Madre es prácticamente nulo, mientras que el de Fiestas Patrias es aún más fuerte de lo que sugería la comparación ingenua.

11. Discrepancia con otro grupo en la hipótesis descuento-cantidad

Otro grupo del curso reportó un coeficiente positivo y altamente significativo para la relación descuento-cantidad (agregando a nivel de boletoa), mientras que el análisis a nivel de línea de producto arrojaba un coeficiente negativo y no significativo. Se verificó que ambos resultados son matemáticamente consistentes entre sí y correctos: la diferencia se explica por el nivel de agregación (ver sección 3.3.2). Al reproducir el análisis del otro grupo

(agregando por boleta) sobre los propios datos, se obtuvo prácticamente el mismo coeficiente (+0,0476 vs. su +0,0477).

5. Limitaciones y extrapolabilidad

¿Es el modelo de regresión extrapolable? No de forma directa, por varias razones concurrentes que se documentan a continuación.

- **Variable omitida más relevante:** el R^2 de 0,25 indica que las tres variables usadas (canal, unidades_producto_boleta, pct_descuento) explican solo una cuarta parte de la variación del monto. Falta, casi con certeza, el producto/precio específico (SKU tiene demasiadas categorías para incluirlo directamente en un OLS simple), que probablemente sea el determinante más importante del monto real.
- **pct_descuento como proxy, no como causa:** según se discutió en 3.3.4, el coeficiente de pct_descuento probablemente refleja que los productos más caros reciben descuentos porcentuales mayores, no que dar más descuento cause un monto más alto. El modelo no permite separar causalidad de esta asociación con los datos disponibles.
- **Heterocedasticidad confirmada:** el test de Breusch-Pagan rechaza el supuesto de varianza constante de los residuos, por lo que los errores estándar reportados (no robustos) deben interpretarse con cautela; una extensión natural es reajustar el modelo con errores estándar robustos a heterocedasticidad (HC3).
- **Ventana temporal acotada y con artefactos:** el dataset cubre solo nov-2023 a dic-2024, con 149 días sin datos y un crecimiento de actividad poco realista tras abril de 2024 (sección 3.2.6). No es posible generalizar conclusiones de tendencia a otros períodos sin volver a entrenar el modelo con datos más representativos.
- **Sesgo de muestreo por diseño del dataset:** dado que el dataset es sintético y presenta patrones no completamente realistas (crecimiento explosivo, amplitud semanal extrema), las conclusiones de negocio deben tratarse como ilustrativas del método, no como recomendaciones operativas definitivas para una farmacia real.

Trabajo futuro

- Modelos predictivos más flexibles (Random Forest, XGBoost) para capturar interacciones no lineales que un OLS simple no puede representar.
- Incorporar el producto (o una categorización agregada del mismo) como predictor, posiblemente mediante técnicas de reducción de dimensionalidad o embeddings categóricos.
- Modelos de series de tiempo dedicados (ARIMA/SARIMA, Prophet) sobre el período confiable (post abril 2024) para pronóstico de demanda.
- Segmentación de clientes (clustering, p.ej. K-means sobre variables RFM) para complementar el análisis de frecuencia de compra.
- Repetir el benchmark de paralelismo con el dataset completo (no solo una muestra) en un entorno con más núcleos, para caracterizar mejor el punto de quiebre entre paralelismo serial y distribuido.

6. Conclusiones

El presente análisis permite extraer conclusiones sustantivas tanto en el ámbito comercial como en la dimensión metodológica del procesamiento de datos. En primer lugar, la evaluación de los canales de venta online (APP y WEB) frente a las sucursales físicas demuestra que no existe evidencia estadística sólida para afirmar que los primeros generen un ticket promedio distinto al canal tradicional. Este hallazgo sugiere que cualquier directriz comercial cimentada sobre el supuesto de que el canal digital tracciona transacciones de mayor volumen orgánico debe ser rigurosamente reevaluada. No obstante, se logró constatar que los canales no presenciales (APP, WEB y CCT) aplican, en promedio, niveles de descuento aproximadamente un 10% superiores a los de las tiendas físicas. Esta discrepancia representa el hallazgo de negocio más accionable del estudio, brindando una oportunidad inmediata para optimizar la rentabilidad de las estrategias promocionales digitales.

En segundo lugar, se descarta la existencia de una relación empírica de utilidad práctica entre la magnitud del descuento aplicado y la cantidad de unidades adquiridas por los clientes (evidenciado por un $R^2 \approx 0,0001$). Esta carencia de poder explicativo se mantiene inalterable sin importar el nivel de agregación utilizado para la medición. Adicionalmente, el análisis de la componente temporal reveló la presencia de una estacionalidad semanal fuerte y consistente, caracterizada por picos de demanda durante los días lunes y jueves. Si bien la identificación de este patrón resulta teóricamente invaluable para la planificación operativa, su magnitud volumétrica debe ser validada contra datos reales de negocio antes de utilizarse como insumo para el dimensionamiento de personal o la gestión de inventarios.

Finalmente, desde una perspectiva metodológica y estructural, el estudio pone en evidencia una colinealidad severa entre las variables de local y canal, dado que el 99,9% de los locales operan bajo una única modalidad de venta. Cualquier esfuerzo futuro orientado a la construcción de modelos predictivos deberá corregir esta redundancia para evitar sesgos en las estimaciones. Como corolario técnico, cabe destacar que la arquitectura implementada resultó en un pipeline analítico reproducible de extremo a extremo (mediante el control de semillas y ejecución parametrizada). El proyecto documenta de forma empírica y explícita bajo qué escenarios el procesamiento paralelo y distribuido aporta una ventaja real de rendimiento, constituyendo un resultado metodológico que resulta tan relevante para la disciplina computacional como los propios hallazgos extraídos de los datos.

7. Referencias teóricas

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Conference Proceedings ,fundamenta por qué el speedup del paralelismo tiene un límite superior dado por la fracción no paralelizable del trabajo, relevante para interpretar el benchmark de la sección 3.1.2.
- Gustafson, J. L. (1988). Reevaluating Amdahl's Law. Communications of the ACM, perspectiva complementaria: a mayor volumen de datos, mayor el beneficio relativo del paralelismo, consistente con la diferencia observada entre la muestra chica (300k filas, Dask pierde) y el dataset completo (3,24M filas, particionado por bloques).
- Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). Biometrika, 52(3/4), 591-611.
- Lilliefors, H. W. (1967). On the Kolmogorov-Smirnov test for normality with mean and variance unknown. Journal of the American Statistical Association, 62(318), 399-402.
- Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. Annals of Mathematical Statistics, 18(1), 50-60.
- Kruskal, W. H., & Wallis, W. A. (1952). Use of ranks in one-criterion variance analysis. Journal of the American Statistical Association, 47(260), 583-621.
- Spearman, C. (1904). The proof and measurement of association between two things. American Journal of Psychology, 15(1), 72-101.
- Breusch, T. S., & Pagan, A. R. (1979). A simple test for heteroscedasticity and random coefficient variation. Econometrica, 47(5), 1287-1294.
- Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). STL: A seasonal-trend decomposition procedure based on loess. Journal of Official Statistics, 6(1), 3-73.
- Vassilvitskii, S., & Zaharia, M. (proyecto Apache Parquet / DuckDB Labs), el formato columnar Parquet reduce la E/S en cargas analíticas al leer solo las columnas requeridas por la consulta; DuckDB explota esta propiedad para ejecutar agregaciones SQL directamente sobre archivos Parquet sin un paso de carga intermedio.